

BUSN 41210: Financial Analytics
Lecture 2: Linear Regression Models

Dacheng Xiu

Chicago Booth

September 9, 2026

© 2026 Dacheng Xiu, Chicago Booth Restricted Materials. All Rights Reserved.

Outline

1. Simple Linear Regressions
2. Prediction and Linear Regression Model
3. Example: Single Factor Model
4. Sampling Distributions, Central Limit Theorem
5. Confidence Interval, Hypothesis Testing
6. Prediction Interval

Linear Regressions

Y = response or outcome variable

$X_1, X_2, X_3, \dots, X_d$ = explanatory or input variables

The general relationship approximated by:

$$Y = f(X_1, X_2, \dots, X_d) + e$$

And, in particular, a linear relationship is written as:

$$Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_dX_d + e$$

Example: Predicting House Prices

Problem:

- ▶ Predict market price based on observed characteristics

Solution:

- ▶ Look at property sales data where we know the price and some observed characteristics.
- ▶ Build a **decision rule** that predicts price as a function of the observed characteristics.

To keep things super simple, let's focus only on size. The value that we seek to predict is called the **dependent (or output)** variable, and we denote this as

- ▶ Y = price of house (e.g. thousands of dollars)

The variable that we use to guide prediction is the **explanatory (or input)** variable, and this is labelled

- ▶ X = size of house (e.g. thousands of square feet)

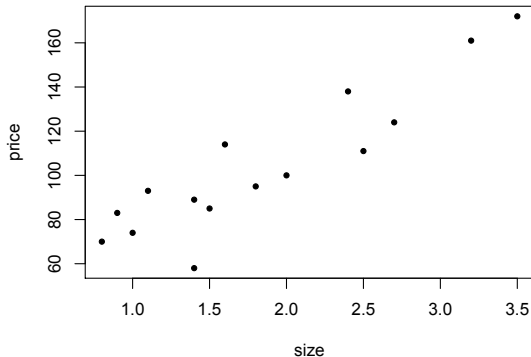
Predicting House Prices

What does this data look like?

Size	Price
0.80	70
0.90	83
1.00	74
1.10	93
1.40	89
1.40	58
1.50	85
1.60	114
1.80	95
2.00	100
2.40	138
2.50	111
2.70	124
3.20	161
3.50	172

Predicting House Prices

It is much more useful to look at a scatterplot

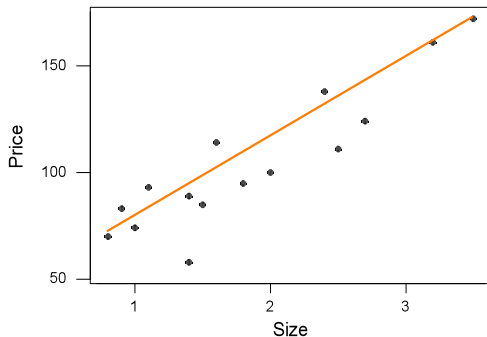


In other words, view the data as points in the $X \times Y$ plane.

Linear Prediction

Appears to be a linear relationship between price and size:

As size goes up, price goes up.



The line shown was fit by the “eyeball” method.

Linear Prediction

Recall that the equation of a line is:

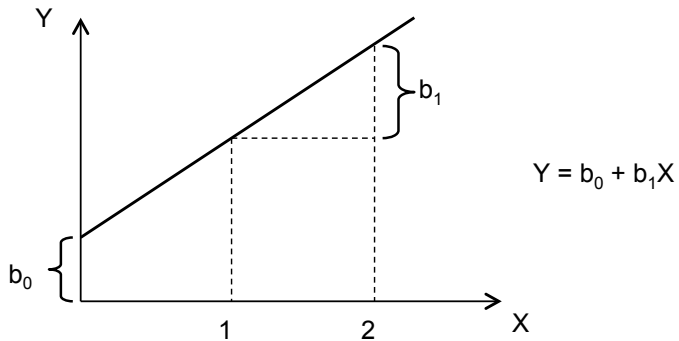
$$Y = b_0 + b_1X$$

Where b_0 is the **intercept** and b_1 is the **slope**.

The intercept value is in units of Y (\$1,000).

The slope is in units of Y *per* units of X (\$1,000/1,000 sq ft).

Linear Prediction



Our “eyeball” line has $b_0 = 35$, $b_1 = 40$.

Linear Prediction

We can now predict the price of a house when we know only the size; just read the value off the line that we've drawn.

For example, given a house with size $X = 2.2$, the predicted price would be

$$\hat{Y} = 35 + 40(2.2 * 1,000 \text{ sq ft}) = \$123,000$$

Note: Conversion from 1,000 sq ft to \$1,000 is done for us by the slope coefficient (b_1)

Linear Prediction

Can we do better than the eyeball method?

We desire a strategy for estimating the slope and intercept parameters in the model

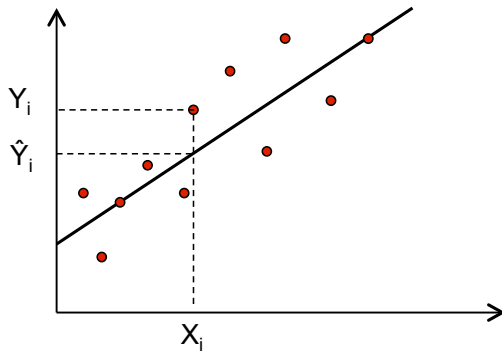
$$\hat{Y} = b_0 + b_1X$$

A reasonable way to fit a line is to minimize the amount by which the **fitted value** differs from the realized value.

This amount is called the **residual**.

Linear Prediction

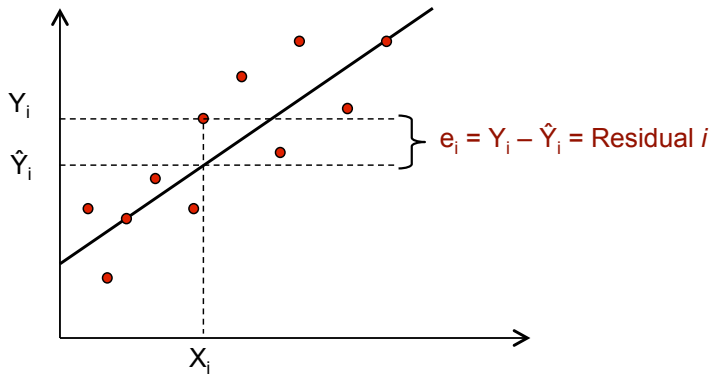
What is the “fitted value”?



The dots are the observed values and the line represents our fitted values given by $\hat{Y}_i = b_0 + b_1 X_1$.

Linear Prediction

What is the “residual” for the i^{th} observation?



We can write $Y_i = \hat{Y}_i + (Y_i - \hat{Y}_i) = \hat{Y}_i + e_i$.

Least Squares

Ideally we want to minimize the size of *all* residuals:

- ▶ If they were all zeros we would have a perfect line.
- ▶ Trade-off between moving closer to some points.

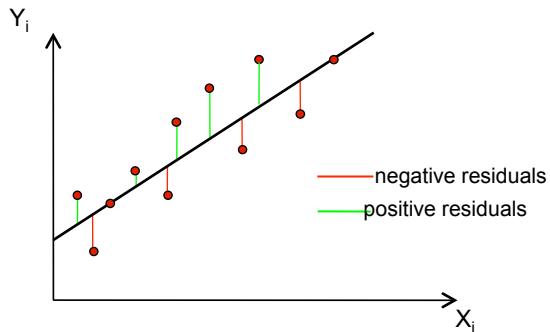
The line fitting process:

- ▶ Give weights to all of the residuals.
- ▶ Minimize the “total” of residuals to get best fit.

Least Squares chooses b_0 and b_1 to minimize

$$\sum_{i=1}^N e_i^2$$

Least Squares



Choose the line to minimize the sum of the squares of the residuals,

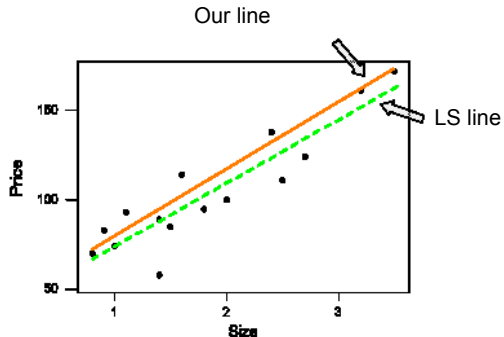
$$\sum_{i=1}^n e_i^2 = \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 = \sum_{i=1}^n (Y_i - [b_0 + b_1 X_i])^2$$

Least Squares

LS chooses a different line from ours:

```
model = smf.ols('Price ~ Size', data=house).fit()  
model.params
```

Intercept	38.884683
Size	35.385963



Least Squares

It turns out that we can use correlations and means to pin down the best fitting line:

```
b1 = np.corrcoef(price, size)[0, 1] * np.std(price, ddof=1) / np.std(size, ddof=1)

b0 = np.mean(price) - np.mean(size) * b1

result = pd.DataFrame({'b0': [b0], 'b1': [b1]})
print(result)
```

0.0s

b0	b1
38.884683	35.385963

Thus our scaled correlation and “means on the line” approach is the same as the least squares estimates.

Least Squares

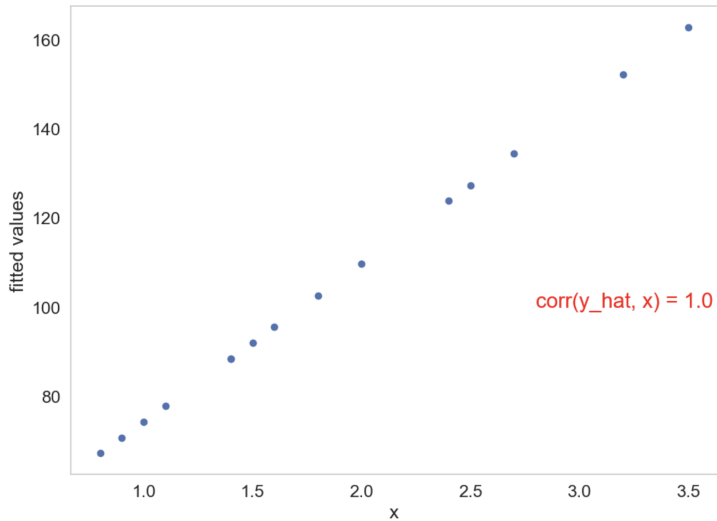
To summarize:

Python's `LinearRegression().fit(X,Y)` from `sklearn.linear_model` or `smf.ols('Y ~ X', data).fit()` from `statsmodels.formula.api` function fits the “least squares” line $\hat{Y} = b_0 + b_1X$ to minimize $\sum_{i=1}^n (Y_i - \hat{Y}_i)^2 = \sum_{i=1}^n e_i^2$.

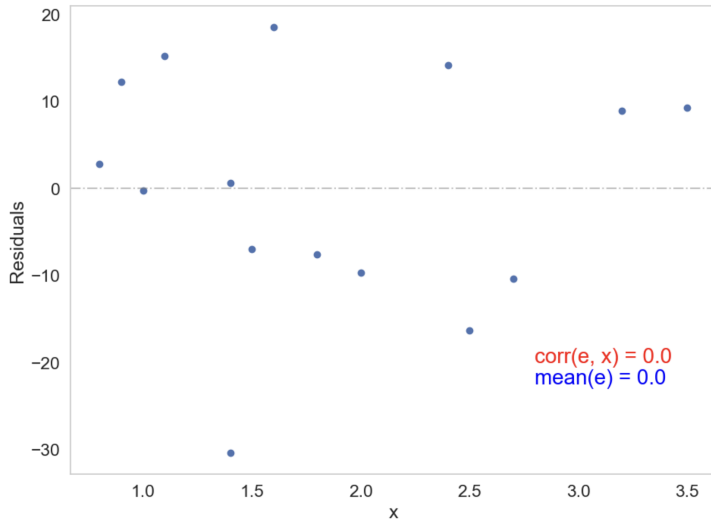
The least squares formulas are

$$b_1 = r_{xy} \frac{s_y}{s_x} \quad \text{and} \quad b_0 = \bar{Y} - b_1 \bar{X}$$

The Fitted Values and X

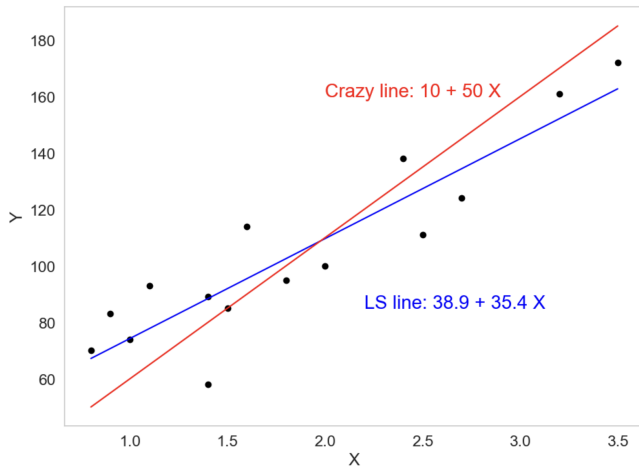


The Residuals and X



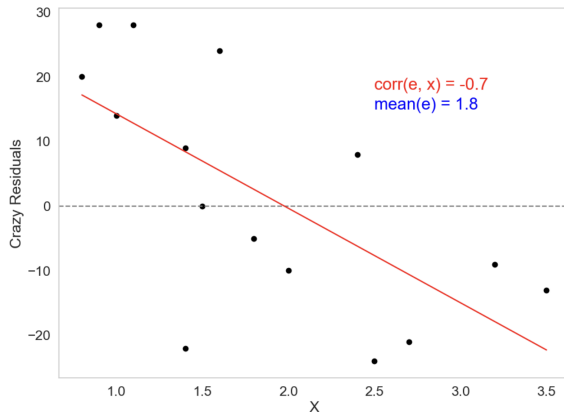
Why?

What is the intuition for the relationship between $\hat{Y}$ and e and X ? Lets consider some “crazy” alternative line:



Fitted Values and Residuals

This is a bad fit! We are underestimating the value of small houses and overestimating the value of big houses.



Clearly, we have left some predictive ability on the table!

Fitted Values and Residuals

As long as the **correlation** between e and X is non-zero, we could always adjust our prediction rule to do better.

We need to exploit all of the predictive power in the X values and put this into $\hat{Y}$, leaving no “ X ness” in the residuals.

In Summary: $Y = \hat{Y} + e$ where:

- ▶ $\hat{Y}$ is “made from X ”; **$\text{corr}(X, \hat{Y}) = 1$ or -1 .**
- ▶ e is uncorrelated to X ; **$\text{corr}(X, e) = 0$.**
- ▶ The average of e is 0; **$E(e) = 0$.**

Another way to derive things

The intercept:

$$\begin{aligned}\frac{1}{n} \sum_{i=1}^n e_i = 0 &\Rightarrow \frac{1}{n} \sum_{i=1}^n (Y_i - b_0 - b_1 X_i) \\ &\Rightarrow \bar{Y} - b_0 - b_1 \bar{X} = 0 \\ &\Rightarrow b_0 = \bar{Y} - b_1 \bar{X}\end{aligned}$$

Another way to derive things

The slope:

Suppose $0 = \text{corr}(e, X)$, then

$$\begin{aligned}\Rightarrow 0 &= \sum_{i=1}^n e_i(X_i - \bar{X}) \\ &= \sum_{i=1}^n (Y_i - b_0 - b_1 X_i)(X_i - \bar{X}) \\ &= \sum_{i=1}^n (Y_i - \bar{Y} - b_1(X_i - \bar{X}))(X_i - \bar{X})\end{aligned}$$

$$\text{Solving } \Rightarrow b_1 = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2} = r_{xy} \frac{s_y}{s_x}$$

Decomposing the Variance

How well does the least squares line explain variation in Y ?

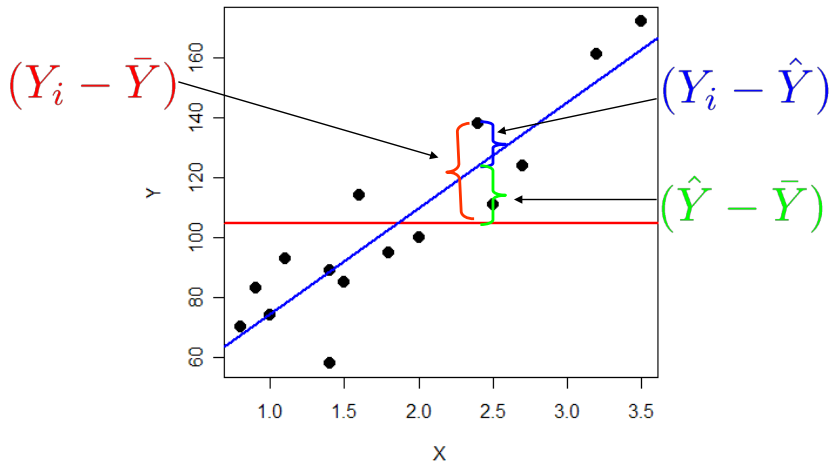
Since $\hat{Y}$ and e are uncorrelated (i.e. $\text{cov}(\hat{Y}, e) = 0$),

$$\text{var}(Y) = \text{var}(\hat{Y} + e) = \text{var}(\hat{Y}) + \text{var}(e)$$

This leads to [ANOVA for regression](#):

$$\sum_{i=1}^n (Y_i - \bar{Y})^2 = \sum_{i=1}^n (\hat{Y}_i - \bar{Y})^2 + \sum_{i=1}^n e_i^2$$

Decomposing the Variance - ANOVA Tables



Decomposing the Variance - ANOVA Tables

$$\underbrace{\sum_{i=1}^n (Y_i - \bar{Y})^2}_{\substack{\text{Total Sum of} \\ \text{Squares} \\ \text{SST}}} = \underbrace{\sum_{i=1}^n (\hat{Y}_i - \bar{Y})^2}_{\substack{\text{Regression SS} \\ \text{SSR}}} + \underbrace{\sum_{i=1}^n e_i^2}_{\substack{\text{Error SS} \\ \text{SSE}}}$$

SSR: Variation in Y explained by the regression line.

SSE: Variation in Y that is left unexplained.

$\text{SSR} = \text{SST} \Rightarrow$ perfect fit.

Be careful of similar acronyms; e.g. SSR for “residual” SS.

A Goodness of Fit Measure: R^2

The **coefficient of determination**, denoted by R^2 , measures goodness of fit:

$$R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

- ▶ $0 < R^2 \leq 1$.
- ▶ The closer R^2 is to 1, the better the fit.

A Goodness of Fit Measure: R^2

An interesting fact: $R^2 = r_{xy}^2$ (i.e., R^2 is squared correlation).

$$\begin{aligned} R^2 &= \frac{\sum_{i=1}^n (\hat{Y}_i - \bar{Y})^2}{\sum_{i=1}^n (Y_i - \bar{Y})^2} \\ &= \frac{\sum_{i=1}^n (b_0 + b_1 X_i - b_0 - b_1 \bar{X})^2}{\sum_{i=1}^n (Y_i - \bar{Y})^2} \\ &= \frac{b_1^2 \sum_{i=1}^n (X_i - \bar{X})^2}{\sum_{i=1}^n (Y_i - \bar{Y})^2} = \frac{b_1^2 s_x^2}{s_y^2} = r_{xy}^2 \end{aligned}$$

No surprise: the higher the sample correlation between X and Y , the better you are doing in your regression.

Summarizing Regression: Back to the House Data

```
> anova_results = anova_lm(model)
```

```
> anova_results
```

	df	sum_sq	mean_sq	F	PR(>F)
Size	1.0	12393.107710	12393.107710	61.998311	0.000003
Residual	13.0	2598.625623	199.894279	NaN	NaN

```
> n = len(model.fittedvalues)
```

```
> var_fitted = np.var(model.fittedvalues, ddof=1) * (n - 1)
```

```
> var_fitted
```

```
Output: 12393.11
```

```
> var_resid = np.var(model.resid, ddof=1) * (n - 1)
```

```
> var_resid
```

```
Output: 2598.63
```

- ▶ The **Residuals** SS is our Error SS (**SSE**).
- ▶ The **size** SS is our Regression SS (**SSR**).

Summarizing Regression: Back to the House Data

```
> model.summary()
```

```

                        OLS Regression Results
=====
Dep. Variable:          Price    R-squared:                0.827
Model:                  OLS      Adj. R-squared:             0.813
Method:                 Least Squares    F-statistic:            62.00
Date:                   Fri, 06 Sep 2024    Prob (F-statistic):      2.66e-06
Time:                   13:38:37    Log-Likelihood:          -59.944
No. Observations:       15    AIC:                      123.9
Df Residuals:           13    BIC:                      125.3
Df Model:                1
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	38.8847	9.094	4.276	0.001	19.238	58.531
Size	35.3860	4.494	7.874	0.000	25.677	45.095

```
> var_price = np.var(house['Price'], ddof=1)
> np.var(model.fittedvalues, ddof=1) / var_price
```

```
Output: 0.826662763718887
```

Prediction and the Modeling Goal

A prediction rule is any function where you input X and it outputs $\hat{Y}$ as a predicted response at X .

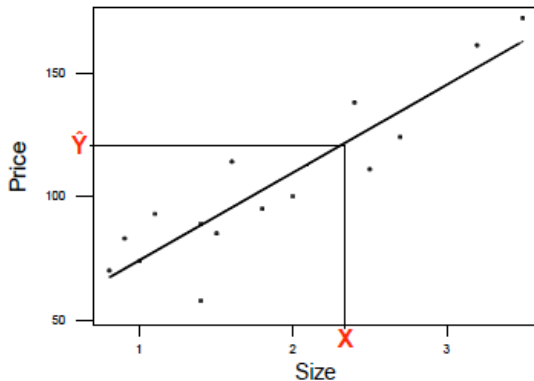
The least squares line is a prediction rule:

$$\hat{Y} = f(X) = b_0 + b_1X$$

Prediction and the Modeling Goal

$\hat{Y}$ is not going to be a perfect prediction.

We need to devise a notion of **prediction accuracy**.



Prediction and the Modeling Goal

There are two things that we want to know:

- ▶ What value of Y can we expect for a given X ?
- ▶ How sure are we about this prediction? Or how different could Y be from what we expect?

Our goal is to measure the accuracy of our prediction **how much uncertainty there is in the prediction**. One method is to specify a range of Y values that are likely, given an X value.

Prediction Interval: probable range for Y -values given X

Prediction and the Modeling Goal

Key Insight: To construct a prediction interval, we will have to assess the likely range of residual values corresponding to a Y value that has not yet been observed!

We will build a **probability model** (e.g., normal distribution).

Then we can say something like “with 95% probability the residuals will be no less than -\$28,000 or larger than \$28,000”.

We must also acknowledge that the “fitted” line may be fooled by particular realizations of the residuals.

The Simple Linear Regression Model

The power of statistical inference comes from the ability to make precise statements about the accuracy of the prediction.

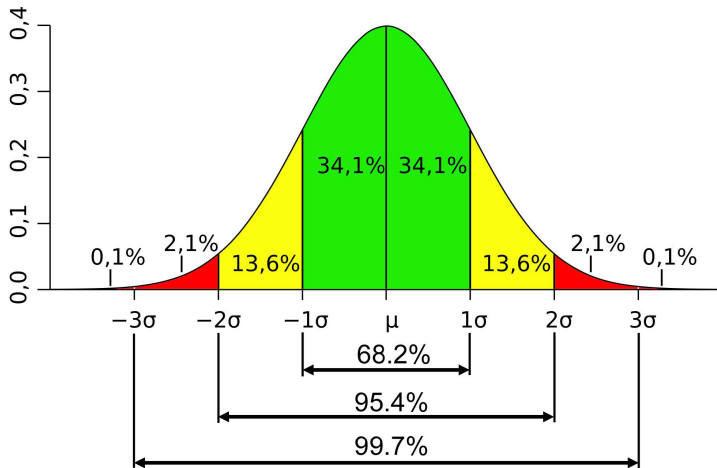
In order to do this we must invest in a **probability model**.

Simple Linear Regression Model: $Y = \beta_0 + \beta_1 X + \varepsilon$

$$\varepsilon \sim N(0, \sigma^2)$$

The error term ε is independent “idiosyncratic noise”.

Normal Distribution and the 2σ Rule



Independent Normal Additive Error

Why do we have $\varepsilon \sim N(0, \sigma^2)$?

- ▶ $\mathbb{E}[\varepsilon] = 0 \Leftrightarrow \mathbb{E}[Y | X] = \beta_0 + \beta_1 X$
($\mathbb{E}[Y | X]$ is “conditional expectation of Y given X ”).
- ▶ Many things are close to Normal (central limit theorem).
- ▶ It works! This is a very robust model for the world.

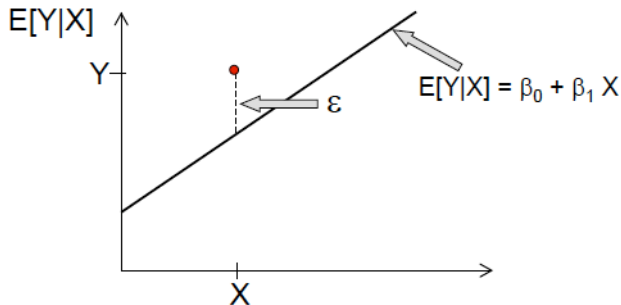
We can think of $\beta_0 + \beta_1 X$ as the “true” regression line.

The Regression Model and our House Data

Think of $\mathbb{E}[Y|X]$ as the expected price of houses with size X :

Some houses could have a higher than expected value, some lower, and the true line tells us what to expect on average.

The error ε represents the influence of factors other than X .

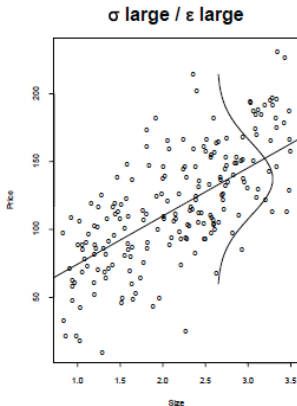
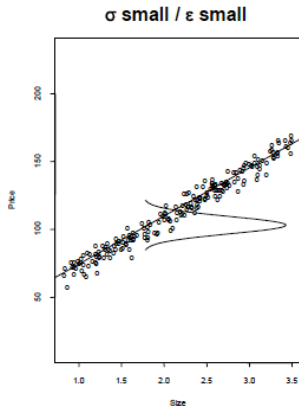


Conditional Distributions

The conditional distribution for Y given X is Normal:

$$Y|X \sim N(\beta_0 + \beta_1 X, \sigma^2).$$

Notice how σ controls dispersion:



Conditional vs Marginal Distributions

More on the conditional distribution:

$$Y|X \sim N(\mathbb{E}[Y|X], \text{Var}(Y|X)).$$

- ▶ Mean is $\mathbb{E}[Y|X] = \mathbb{E}[\beta_0 + \beta_1 X + \varepsilon|X] = \beta_0 + \beta_1 X$.
- ▶ Variance is $\text{Var}(Y|X) = \text{Var}(\beta_0 + \beta_1 X + \varepsilon|X) = \text{Var}(\varepsilon) = \sigma^2$.
- ▶ $\sigma^2 < \text{Var}(Y)$ if X and Y are correlated.

And the ANOVA for regression:

- ▶ The bigger $[1 - \sigma^2/\text{Var}(Y)]$, the more X matters!

Prediction Intervals with the True Model

You are told (without looking at the data) that

$$\beta_0 = 40; \beta_1 = 45; \sigma = 10$$

and you are asked to predict price of a 1500 square foot house.

What do you know about Y from the model?

$$\begin{aligned} Y &= 40 + 45(1.5) + \varepsilon \\ &= 107.5 + \varepsilon \end{aligned}$$

Thus our prediction for price is

$$Y \sim N(107.5, 10^2)$$

Prediction Intervals with the True Model

The model says that the mean value of a 1500 sq. ft. house is \$107,500 and that deviation from mean is within $\approx$ \$20,000.

We are 95% sure that

- ▶ $-20 < \varepsilon < 20$
- ▶ $\$87,500 < Y < \$127,500$

In general, the 95 % Prediction Interval is $PI = \beta_0 + \beta_1 X \pm 2\sigma$.

Next, we will learn about how to incorporate another source of risk: uncertainty in the β_0 , β_1 and σ .

Recall - Estimation for the SLR Model

SLR assumes every observation in the dataset was generated by the model:

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i$$

This is a model for the conditional distribution of Y given X .

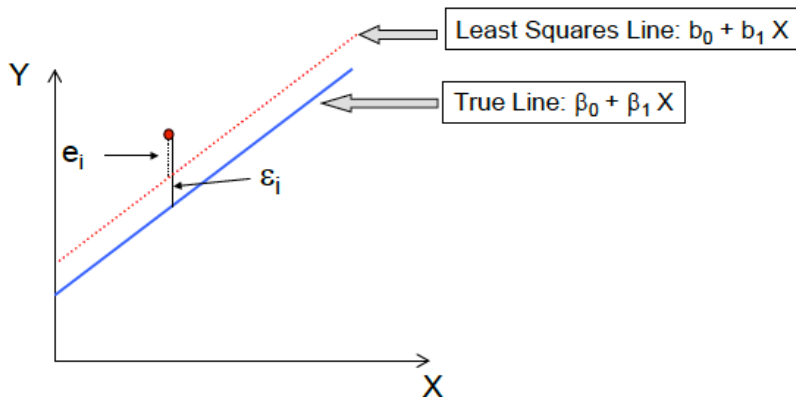
We use Least Squares to estimate β_0 and β_1 :

$$\hat{\beta}_1 = b_1 = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2}$$

$$\hat{\beta}_0 = b_0 = \bar{Y} - b_1 \bar{X}$$

Recall - Estimation for the SLR Model

NOTE!!: β_0 is not b_0 , β_1 is not b_1 and ε_i is not e



Single Factor (Index) Model (Statistical Model)

- ▶ The model relates return of asset i , r_i to the “market” return, r_M .
- ▶ In particular, the relationship is given by the **regression** model
 $r_i - r_f = \alpha + \beta(r_M - r_f) + \varepsilon$ with observations at times $t = 1 \dots T$ (and where $[\alpha, \beta] \equiv [\beta_0, \beta_1]$).

Single Factor Model

- Notice that the covariance between two assets i and j is:

$$\text{cov}(r_i, r_j) = \beta_i \beta_j \sigma_M^2$$

The total number of estimates needed for covariances is:

$$\begin{aligned} & n \text{ estimates of } \beta_i \\ & +1 \text{ estimate for the variance } \sigma_M^2 \end{aligned}$$

- BTW, there is a nice interpretation of the total risk of a firm:

Total risk = Systematic risk + Firm-specific risk

$$\sigma_i^2 = \beta_i^2 \sigma_M^2 + \sigma^2(\varepsilon)$$

Capital Asset Pricing Model (Economic Model)

- ▶ The model implies that $E(r_i) - r_f = \beta_i(E(r_M) - r_f)$
- ▶ In particular, it predicts that $\alpha_i = 0$, for any asset i .
- ▶ When asset i is a mutual fund, $[\alpha_i, \beta_i]$ can be used as a performance benchmark for fund managers.

CAPM Example

Data:

- ▶ Weekly stock prices from 1/1/2010 to 12/31/2011 of several firms listed in US stock markets, including AAPL, AIG, C, GE, GOOG, GS, and MMM.
- ▶ Weekly annualized US 1-month T-Bill rates from 1/1/2010 to 12/31/2011 in percentage.

Problem:

- ▶ Calculate alphas and betas.

CAPM Example

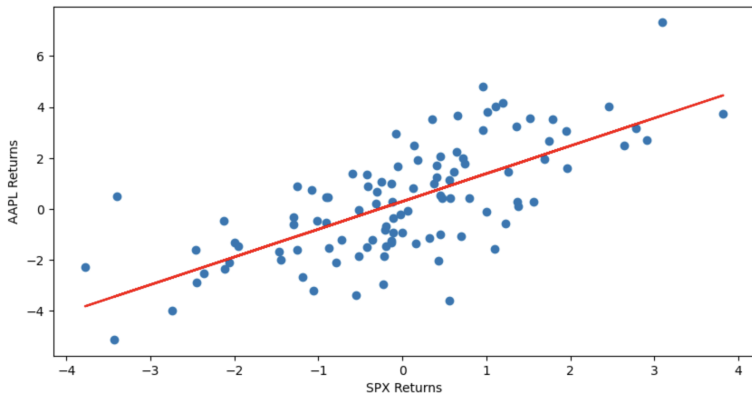
What does this data look like?

```
capm = pd.read_csv('./beta.csv', index_col='Date')
Tbill = capm['TBILL'].copy()
capm.head(10)
```

	AAPL	AIG	C	GE	GOOG	GS	MMM	SPX	TBILL
Date									
1/1/10	210.732	25.1146	33.0721	14.2195	619.980	165.3277	78.5626	1115.10	0.03
1/8/10	211.980	24.5784	35.8697	15.6011	602.020	170.6839	80.1306	1144.98	0.03
1/15/10	205.930	23.5062	34.1712	15.4507	580.000	161.7732	79.2278	1136.03	0.02
1/22/10	197.750	22.1993	32.4726	15.1406	550.010	150.9139	77.4317	1091.76	0.03
1/29/10	192.063	20.2977	33.1720	15.1124	529.944	145.6263	76.4909	1073.87	0.02
2/5/10	195.460	18.7731	32.1729	14.8398	531.290	150.9531	74.6378	1066.19	0.04
2/12/10	200.380	22.4674	31.7732	14.6143	533.120	150.7279	75.2460	1075.51	0.05
2/19/10	201.670	22.2245	34.1712	15.1969	540.763	152.9311	77.9784	1109.17	0.06
2/26/10	204.620	20.7501	33.9713	15.1878	526.800	153.4408	76.6680	1104.49	0.08
3/5/10	218.950	23.5229	34.9705	15.4620	564.210	164.0693	78.8585	1138.70	0.09

CAPM Example

Let's look at AAPL:



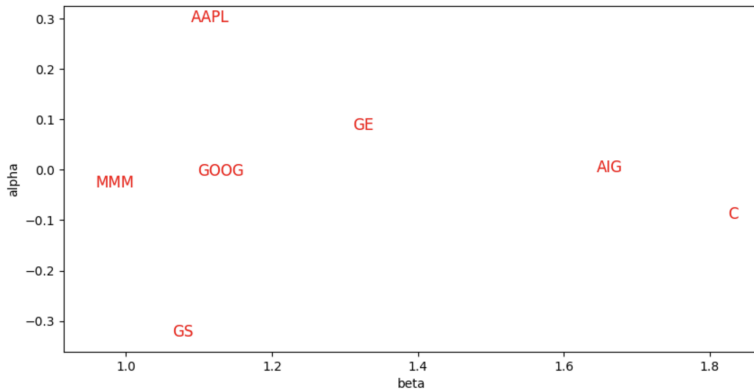
CAPM Example - Summary of Regression

```
> model_capm = smf.ols('AAPL ~ SPX', data=capm).fit()  
> model_capm.summary()
```

OLS Regression Results						
=====						
Dep. Variable:	AAPL		R-squared:	0.482		
Model:	OLS		Adj. R-squared:	0.477		
Method:	Least Squares		F-statistic:	94.96		
Date:	Thu, 05 Sep 2024		Prob (F-statistic):	2.99e-16		
Time:	11:46:27		Log-Likelihood:	-195.12		
No. Observations:	104		AIC:	394.2		
Df Residuals:	102		BIC:	399.5		
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

Intercept	0.2943	0.156	1.882	0.063	-0.016	0.605
SPX	1.0893	0.112	9.745	0.000	0.868	1.311

CAPM Example



Estimation of Error Variance

Recall that $\varepsilon_i \stackrel{iid}{\sim} N(0, \sigma^2)$, and that σ drives the width of the prediction intervals:

$$\sigma^2 = \text{var}(\varepsilon_i) = \mathbb{E}[(\varepsilon_i - \mathbb{E}[\varepsilon_i])^2] = \mathbb{E}[\varepsilon_i^2]$$

A sensible strategy would be to estimate the average for squared errors with the sample average squared residuals:

$$s^2 = \frac{1}{N} \sum_{i=1}^n e_i^2$$

Estimation of Error Variance

In order to obtain an unbiased estimator of σ^2 , what is the right value for N in the denominator:

$$s^2 = \frac{1}{n - p} \sum_{i=1}^n e_i^2 = \frac{SSE}{n - 2}$$

(p is the number of regression coefficients; i.e. 2 for $\beta_0 + \beta_1$).

We have $n - p$ degrees of freedom because p ($= 2$) have been “used up” in the estimation of b_0 and b_1 .

We usually use $s = \sqrt{SSE/(n - p)}$, in the same units as Y .

Degrees of Freedom

Degrees of Freedom is the number of times you get to observe useful information about the variance you're trying to estimate.

For example, consider $SST = \sum_{i=1}^n (Y_i - \bar{Y})^2$:

- ▶ If $n = 1$, $\bar{Y} = Y_1$ and $SST = 0$: since Y_1 is “used up” estimating the mean, we haven't observed any variability!
- ▶ For $n > 1$, we've only had $n - 1$ chances for deviation from the mean, and we estimate $s_y^2 = SST / (n - 1)$.

In regression with p coefficients (e.g., $p = 2$ in SLR), you only get $n - p$ real observations of variability $\Rightarrow DoF = n - p$.

Estimation of Error Variance

```
> anova_lm(model_capm)
```

	df	sum_sq	mean_sq	F	PR(>F)
SPX	1.0	241.601109	241.601109	94.964514	2.988360e-16
Residual	102.0	259.500228	2.544120	NaN	NaN

```
> np.sqrt(anova_result.mean_sq[1])
```

Output: 1.5950

```
> np.sqrt(model_capm.scale)
```

Output: 1.5950

```
> np.sqrt(sum(model_capm.resid ** 2) / model_capm.df_resid)
```

Output: 1.5950

Sampling Distribution of Least Squares Estimates

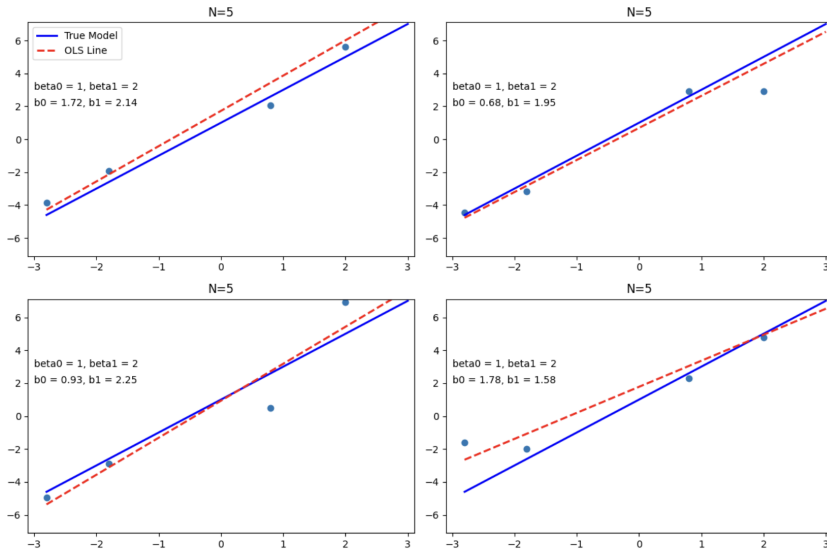
How much do our estimates depend on the particular random sample that we happen to observe? Imagine:

- ▶ Randomly draw different samples of the same size.
- ▶ For each sample, compute the estimates b_0 , b_1 , and s .

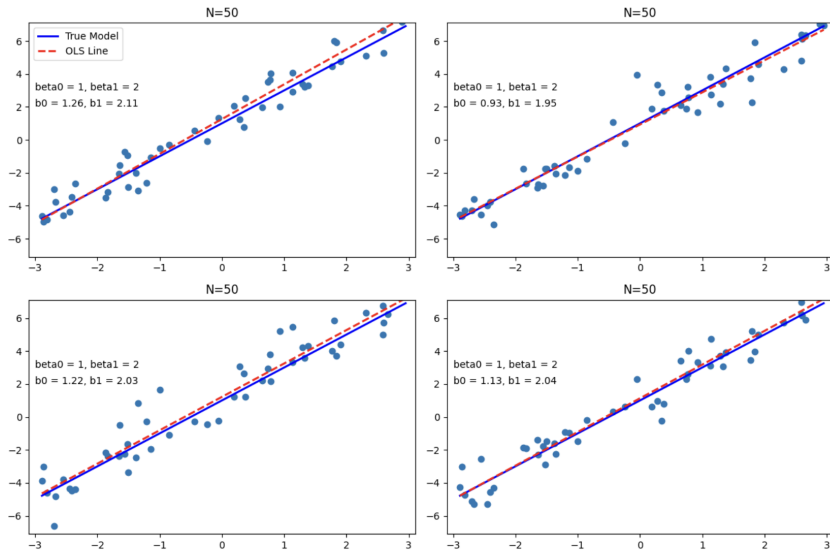
If the estimates don't vary much from sample to sample, then it doesn't matter which sample you happen to observe.

If the estimates do vary a lot, then it matters which sample you happen to observe.

Sampling Distribution of Least Squares Estimates



Sampling Distribution of Least Squares Estimates



Sampling Distribution of Least Squares Estimates

LS lines are much closer to the true line when $n = 50$.

For $n = 5$, some lines are close, others aren't:

we need to get "lucky"

Review: Sampling Distribution of Sample Mean

Step back for a moment and consider the mean for an *iid* sample of n observations of a random variable $\{X_1, \dots, X_n\}$

Suppose that $\mathbb{E}(X_i) = \mu$ and $\text{var}(X_i) = \sigma^2$

$$\blacktriangleright \mathbb{E}(\bar{X}) = \frac{1}{n} \sum \mathbb{E}(X_i) = \mu$$

$$\blacktriangleright \text{var}(\bar{X}) = \text{var}\left(\frac{1}{n} \sum X_i\right) = \frac{1}{n^2} \sum \text{var}(X_i) = \frac{\sigma^2}{n}$$

If X is normal, then $\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right)$.

If X is not normal, we have the central limit theorem!

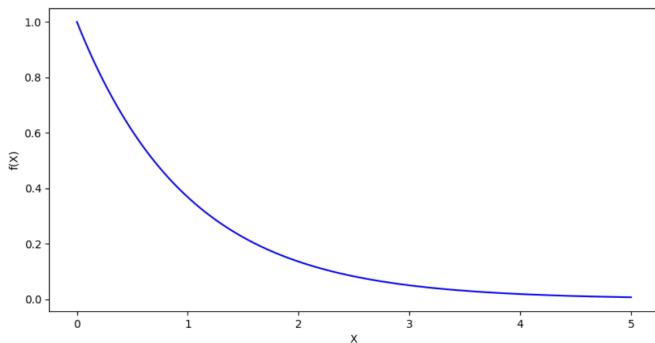
Central Limit Theorem

Simple CLT states that for *iid* random variables, X , with mean μ and variance σ^2 , the distribution of the sample mean becomes normal as the number of observations, n , gets large.

That is, $\bar{X} \rightarrow_n N(\mu, \sigma^2/n)$, and sample averages tend to be normally distributed in large samples.

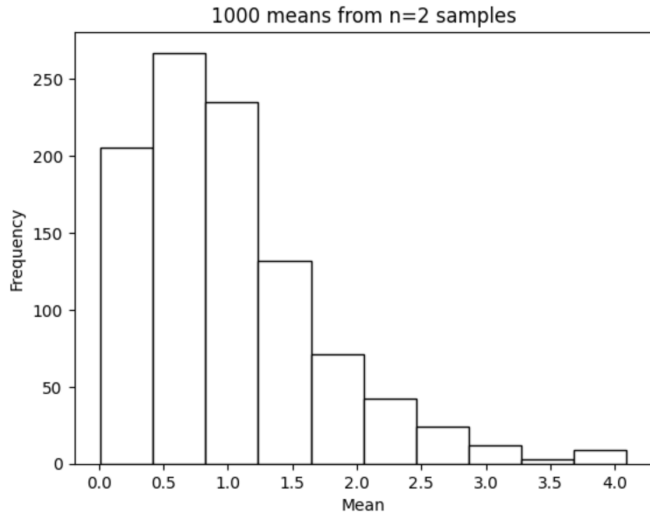
Central Limit Theorem

Exponential random variables don't look very normal:

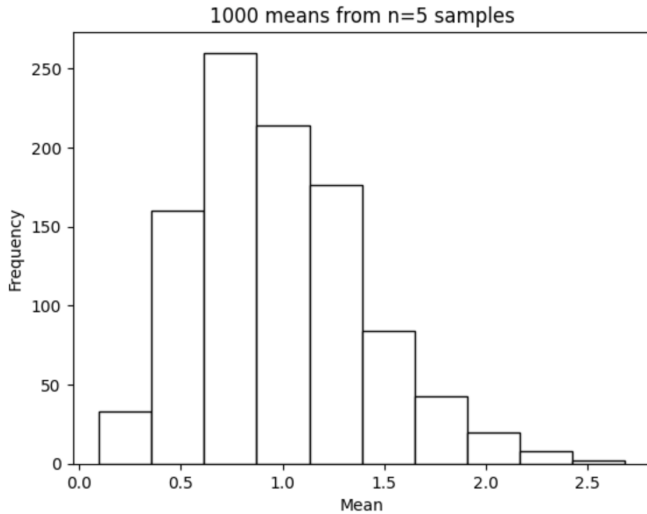


$$\mathbb{E}[X] = 1 \text{ and } \text{var}(X) = 1.$$

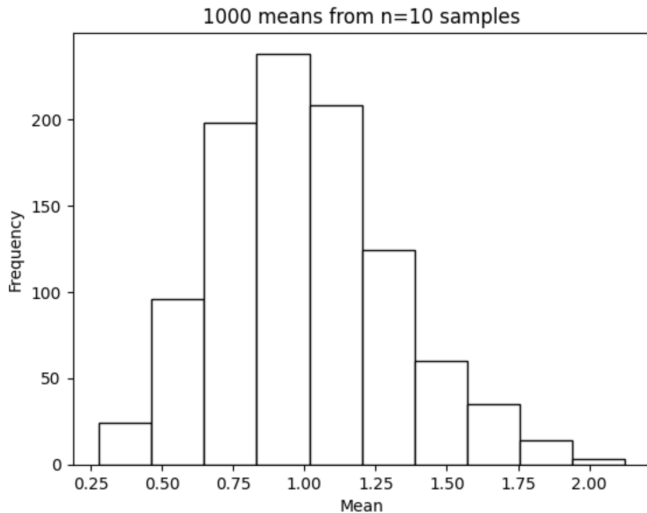
Central Limit Theorem



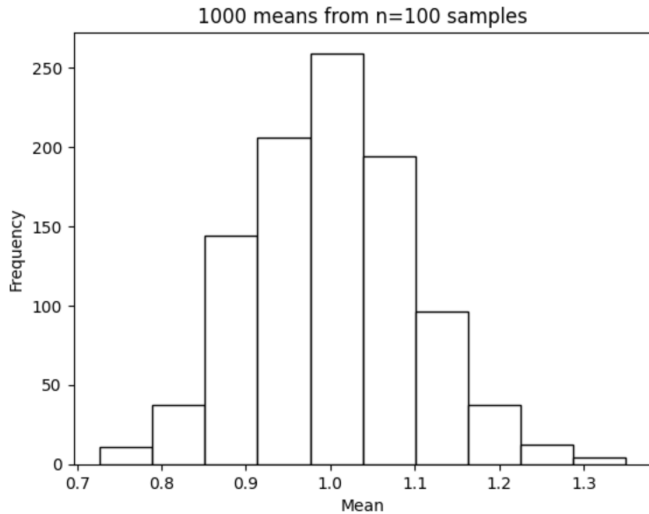
Central Limit Theorem



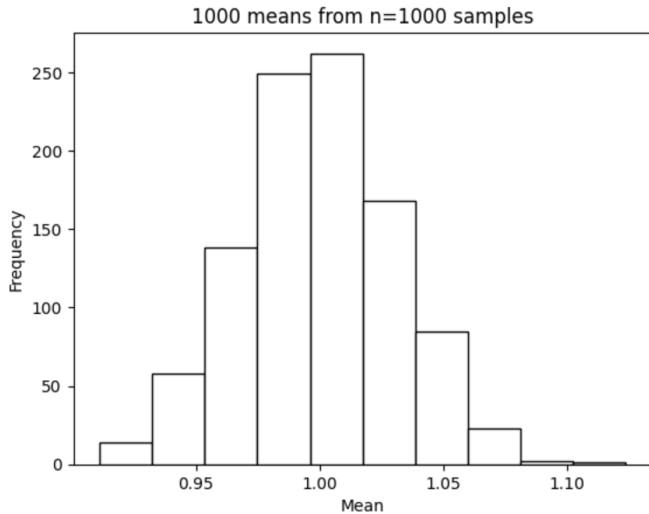
Central Limit Theorem



Central Limit Theorem



Central Limit Theorem



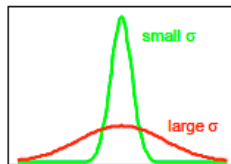
Sampling Distribution of b_1

The sampling distribution of b_1 describes how estimator $b_1 = \hat{\beta}_1$ varies over different samples with the X values fixed.

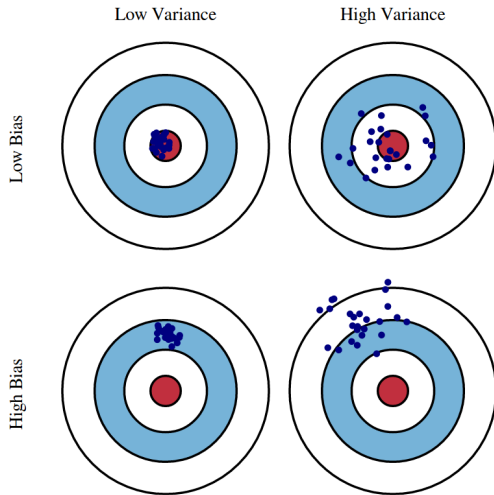
It turns out that b_1 is normally distributed: $b_1 \sim N(\beta_1, \sigma_{b_1}^2)$.

- ▶ b_1 is unbiased: $\mathbb{E}[b_1] = \beta_1$.
- ▶ Sampling sd σ_{b_1} determines precision of b_1 .

The variance term determines how close the estimate will be to the true value.
Remember: large σ is bad!



Bias and Variance



Sampling Distribution of b_1

Can we intuit what should be in the formula for σ_{b_1} ?

- ▶ How should σ figure in the formula?
- ▶ What about n ?
- ▶ Anything else?

$$\sigma_{b_1}^2 = \text{var}(b_1) = \frac{\sigma^2}{\sum (X_i - \bar{X})^2} = \frac{\sigma^2}{(n-1)s_x^2}$$

Three Factors:

sample size (n), error variance ($\sigma^2 = \sigma_\varepsilon^2$), and X -spread (s_x).

Optional: Derivation of $\text{Var}(b_1)$

First off, $b_1 = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sum (X_i - \bar{X})^2} = \frac{\sum (X_i - \bar{X})Y_i}{\sum (X_i - \bar{X})^2}$

since $\sum (X_i - \bar{X}) = \sum X_i - n\bar{X} = 0$. Thus

$$b_1 = \sum \frac{X_i - \bar{X}}{D} Y_i = \sum w_i Y_i$$

where $D = \sum (X_i - \bar{X})^2$ is the sum of squares for X . Finally,

$$\text{var}(b_1) = \sum w_i^2 \text{var}(Y_i | X_i) = \sigma^2 \sum w_i^2 = \sigma^2 \frac{D}{D^2} = \frac{\sigma^2}{D}$$

(Note: b_1 is more heavily weighted by X_i that are far from $\bar{X}$.)

Sampling Distribution of b_0

The intercept is also **normal** and **unbiased**: $b_0 \sim N(\beta_0, \sigma_{b_0}^2)$.

$$\sigma_{b_0}^2 = \text{var}(b_0) = \sigma^2 \left(\frac{1}{n} + \frac{\bar{X}^2}{(n-1)s_x^2} \right)$$

What is the intuition here?

$$\text{var}(\bar{Y} - \bar{X}b_1) = \text{var}(\bar{Y}) + \bar{X}^2 \text{var}(b_1) + 2\text{cov}(\bar{Y}, b_1)$$

$\bar{Y}$ and b_1 are uncorrelated because the slope (b_1) is invariant if you shift the data up or down ($\bar{Y}$).

Joint Distribution of b_0 and b_1

We know that b_0 and b_1 *can be* dependent, which means that an estimation error in the slope is correlated with the estimation error in the intercept.

$$\text{cov}(b_0, b_1) = -\sigma^2 \left(\frac{\bar{X}}{(n-1)s_x^2} \right)$$

- ▶ Usually ($\bar{X} > 0$), if the slope estimate is too high, the intercept estimate is too low (negative correlation).
- ▶ The correlation **decreases** with more X spread (s_x^2).

Estimated Variance

We estimate variation with “sample standard deviations”:

$$s_{b_1} = \sqrt{\frac{s^2}{(n-1)s_x^2}} \quad s_{b_0} = \sqrt{s^2 \left(\frac{1}{n} + \frac{\bar{X}^2}{(n-1)s_x^2} \right)}$$

Recall that $s = \sqrt{\sum e_i^2 / (n - p)}$ is the estimator for $\sigma = \sigma_\varepsilon$.
Hence, $s_{b_1} = \hat{\sigma}_{b_1}$ and $s_{b_0} = \hat{\sigma}_{b_0}$ are estimated coefficient sd's.

A high level of info/precision/accuracy means small s_b values.

What Is a Standard Error, Really?

We just derived $\text{var}(b_1) = \sigma^2 / \sum (x_i - \bar{x})^2$ and called its square root the standard error. **Where does that number actually come from?**

The definition is a thought experiment: if the world were run again — same market returns, different luck — how different would your b_1 be?

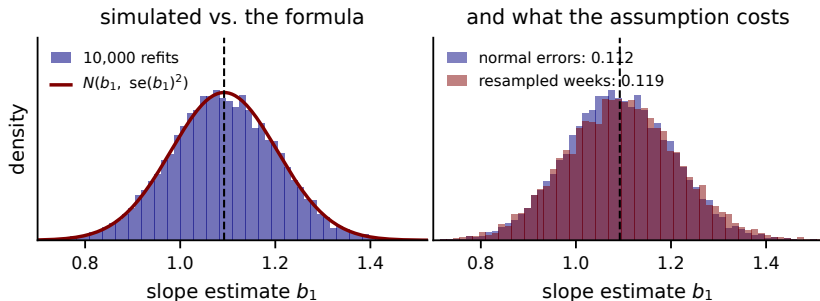
We can stop imagining and just do it. We fitted AAPL on the S&P 500, $n = 104$ weeks, and got

$$b_1 = 1.0923, \quad \text{se}(b_1) = 0.1116.$$

Now treat that fitted line as if it were the truth, generate 10,000 alternative histories from it, refit all 10,000, and look at the spread.

Before the next slide: how wide will that histogram be? Write down a number. Is the spread of ten thousand refits going to be near 0.11, or nowhere near it?

Two Routes to the Same Number



- ▶ **Route 1, the formula:** one regression, one line of algebra. $\text{se}(b_1) = 0.1116$.
- ▶ **Route 2, brute force:** 10,000 refits on simulated histories. The standard deviation of those 10,000 slopes is 0.1120.
- ▶ **They agree to 0.3%,** and nothing was rigged — the two routes share no arithmetic at all.

And the interval does what it says on the tin: the textbook 95% confidence interval contained the true slope in 95.0% of the ten thousand replications.

That is the whole content of a standard error — not a decoration on a coefficient, but a statement about how much your answer would move if you had been dealt a different hand.

What the Assumption Costs

Route 2 assumed what the formula assumes: errors that are normal, independent, and the same size in every week. Weekly equity returns are not like that — calm months and panics do not have the same variance.

So try a third route that assumes none of it. Resample the 104 actual weeks with replacement, refit, ten thousand times:

formula, textbook	0.1116
simulation, normal errors	0.1120
resampling the actual weeks	0.1190

The textbook standard error is about 7% too small here. Neither number is wrong; they answer the question under different assumptions, and the gap between them is the size of the assumption.

It is why practitioners report “robust” standard errors — and it is the first time in this course that a formula and the data have disagreed. It will not be the last.

Normal and Student's t

Recall what *Student* discovered:

If $\theta \sim N(\mu, \sigma^2)$, but you estimate $\sigma^2 \approx s^2$ based on $n - p$ degrees of freedom, then $\theta \sim t_{n-p}(\mu, s^2)$.

For example:

- ▶ $\bar{Y} \sim t_{n-1}(\mu, s_y^2/n)$.
- ▶ $b_0 \sim t_{n-2}(\beta_0, s_{b_0}^2)$ and $b_1 \sim t_{n-2}(\beta_1, s_{b_1}^2)$

The t distribution is just a **fat-tailed** version of the normal. As $n - p \rightarrow \infty$, our tails get skinny and the t becomes normal.

Standardized Normal and Student's t

We'll also usually standardize things:

$$\frac{b_j - \beta_j}{\sigma_{b_j}} \sim N(0, 1) \implies \frac{b_j - \beta_j}{s_{b_j}} \sim t_{n-2}(0, 1)$$

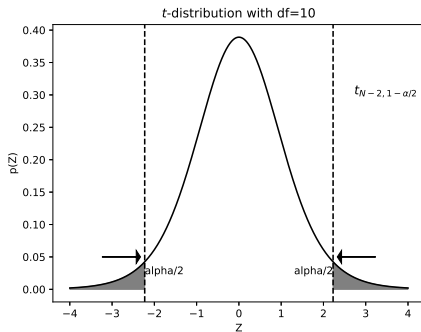
We use $Z \sim N(0, 1)$ and $Z_{n-p} \sim t_{n-p}(0, 1)$ to represent standard random variables.

Notice that the t and normal distributions depend upon assumed values for β_j : this forms the basis for confidence intervals, hypothesis testing, and p -values.

Confidence Intervals and Testing

Suppose Z_{n-p} is distributed $t_{n-p}(0, 1)$. A centered interval is

$$\Pr(-t_{n-p, 1-\alpha/2} < Z_{n-p} < t_{n-p, 1-\alpha/2}) = 1 - \alpha$$



- ▶ Total area under tails = α
- ▶ $\alpha/2$ on each side

Confidence Intervals

Since $b_j \sim t_{n-p}(\beta_j, s_{b_j})$,

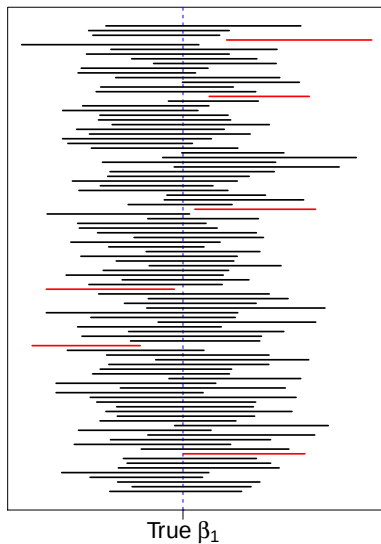
$$\begin{aligned} 1 - \alpha &= P(-t_{n-p,\alpha/2} < \frac{b_j - \beta_j}{s_{b_j}} < t_{n-p,\alpha/2}) \\ &= P(b_j - t_{n-p,\alpha/2}s_{b_j} < \beta_j < b_j + t_{n-p,\alpha/2}s_{b_j}) \end{aligned}$$

Thus $(1 - \alpha)*100\%$ of the time, β_j is within the Confidence Interval:

$$b_j \pm t_{n-p,\alpha/2}s_{b_j}$$

Coverage Probability

$$P\left[\beta_1 \in (b_1 \pm 2s_{b_1})\right] = 95\%$$



More Detail... Confidence Intervals

Why should we care about Confidence Intervals?

- ▶ The confidence interval captures the amount of information in the data about the parameter.
- ▶ The center of the interval tells you what your estimate is.
- ▶ The length of the interval tells you how sure you are about your estimate.

Example: Confidence Intervals

We first choose $\alpha = 0.05$, then we find the cut-offs.

```
> model_capm.conf_int(alpha=0.05)
```

```
95% confidence intervals:
              0              1
Intercept -0.015936  0.604524
SPX        0.867574  1.311002
```

```
> df_resid = n - p
> t025 = stats.t.ppf(0.975, df_resid)
> params = model_capm.params
> stderr = model_capm.bse
> ci_intercept = [params[0] - t025 * stderr[0], params[0] + t025 * stderr[0]]
Output: [-0.015936, 0.604524]
> ci_slope = [params[1] - t025 * stderr[1], params[1] + t025 * stderr[1]]
Output: [0.867574, 1.311002]
```

`scipy.stats.t.ppf` is the Student's t “quantile function”, and
`scipy.stats.t.ppf(prob, df)` returns t_{df} such that $prob = P(Z_{df} < t_{df})$.

Hypothesis Testing

Suppose that we are interested in the slope parameter, β_1 .

For example, is there any evidence in the data to support the existence of a relationship between X and Y?

We can rephrase this in terms of competing hypotheses.

$H_0 : \beta_1 = 0$. Null/safe; implies “no effect” and we ignore X.

$H_1 : \beta_1 \neq 0$. Alternative; leads us to our best guess $\beta_1 = b_1$.

Hypothesis Testing

Assuming that the null hypothesis is true, the decision rule states a range of values for the test statistic to be plausible and a range of values that seem implausible.

Our hypothesis test will either reject or not reject the null hypothesis (the default and usually simpler claim).

If the hypothesis test rejects the null hypothesis, we have statistical support for alternative hypothesis.

Hypothesis Testing

We use b_j for our test about β_j .

- ▶ Reject H_0 when b_j is far from β_j^0 (usually 0).
- ▶ Assume H_0 when b_j is close to β_j^0 .

An obvious tactic is to look at the difference $b_j - \beta_j^0$.

But this measure doesn't take into account the uncertainty in estimating b_j : What we really care about is how many standard deviations b_j is away from β_j^0 .

Hypothesis Testing

The *t*-statistic for this test is

$$z_{b_j} = \frac{b_j - \beta_j^0}{s_{b_j}} = \frac{b_j}{s_{b_j}} \text{ for } \beta_j^0 = 0.$$

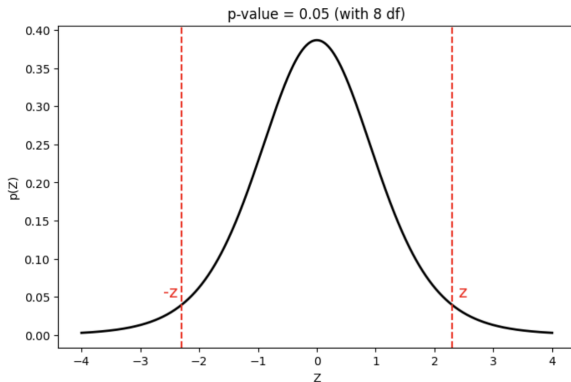
If H_0 is true, this *should* be distributed $z_{b_j} \sim t_{n-p}(0, 1)$.

- ▶ Small $|z_{b_j}|$ leaves us happy with the null β_j^0 .
- ▶ Large $|z_{b_j}|$ (i.e., $>$ about 2) should get us worried!

Hypothesis Testing

Assuming that the null hypothesis is true, what is the probability of observing a value for the test statistic that is at least as extreme as the value that was actually observed?

p-value: $\varphi = P(|Z_{n-p}| > |z_{b_j}|) = 2P(Z_{n-p} > |z_{b_j}|)$



Hypothesis Testing

The formal 2-step approach to hypothesis testing

- ▶ Pick the significance level α (often $1/20 = 0.05$), our acceptable risk (probability) of rejecting a true null hypothesis (we call this a type 1 error).
This α plays the same role as α in CI's.
- ▶ Calculate the p -value, and reject H_0 if $\varphi < \alpha$ (in favor of our best alternative guess; e.g. $\beta_j = b_j$).
If $\varphi > \alpha$, continue working under null assumptions.

This is equivalent to having the rejection region $|z_{b_j}| > t_{n-p,\alpha/2}$.

Example: Hypothesis Testing

Consider again a CAPM regression for the APPL.

Does AAPL have a non-zero intercept?

$H_0 : \beta_0 = 0$ and no expected return in excess of (or below) the fair expected return predicted by the CAPM.

$H_1 : \beta_0 \neq 0$ and AAPL is underpriced or overpriced.

Example: Hypothesis Testing

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.2943	0.156	1.882	0.063	-0.016	0.605
SPX	1.0893	0.112	9.745	0.000	0.868	1.311

```
> b0 = model_capm.params['Intercept']
```

```
Intercept (b0): 0.2943
```

```
> n = len(model_capm.fittedvalues)
```

```
> sb0 = model_capm.bse['Intercept']
```

```
Standard Error of the Intercept (sb0): 0.156
```

```
> t_stat = b0 / sb0
```

```
t_stat: 1.882
```

```
> p_value = 2 * (1 - stats.t.cdf(abs(t_stat), df=n-2))
```

```
p_value: 0.063
```

The t “distribution function” `scipy.stats.t.cdf(z, df)` returns $P(Z_{df} < z)$.

It turns out that we cannot reject the null at $\alpha = .05$.

Example: Hypothesis Testing

Looking at the slope:

$H_0 : \beta_1 = 1$ AAPL is as aggressive as the market.

$H_1 : \beta_1 \neq 1$ and AAPL softens or exaggerates market moves.

Example: Hypothesis Testing

This time, Python's output t/p values are not what we want (why?).

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.2943	0.156	1.882	0.063	-0.016	0.605
SPX	1.0893	0.112	9.745	0.000	0.868	1.311

But we can get the appropriate values easily:

```
> b1 = model_capm.params['SPX']
> n = len(model_capm.fittedvalues)
> sb1 = model_capm.bse['SPX']
> zb1 = (b1 - 1) / sb1
> p_value = 2 * stats.t.cdf(-abs(zb1), df=n-2)
p_value: 0.4262693
```

We thus cannot reject the null at $\alpha = .05$ ($\varphi = .426$).

Prediction

The **conditional prediction problem**: Given covariate X_f and sample data $\{X_i, Y_i\}_{i=1}^n$, predict the “future” observation y_f .

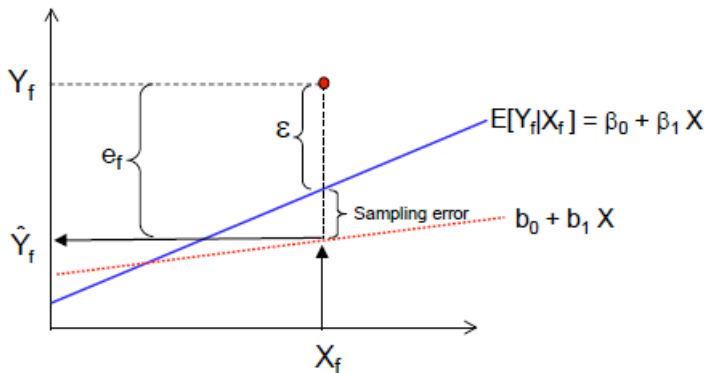
The solution is to use our LS fitted value: $\hat{Y}_f = b_0 + b_1 X_f$.

This is the easy bit. The hard (**and very important!**) part of predict is assessing uncertainty about our predictions.

Prediction

If we use $\hat{Y}_f$, our **prediction error** is

$$e_f = Y_f - \hat{Y}_f = Y_f - b_0 - b_1 X_f$$



Prediction

We can decompose e_f into two sources of error

- ▶ Inherent idiosyncratic randomness (due to ε).
- ▶ Estimation error in the intercept and slope (i.e., discrepancy between our line and “the truth”).

$$\begin{aligned}e_f &= Y_f - \hat{Y}_f = (Y_f - \mathbb{E}[Y_f|X_f]) + \mathbb{E}[Y_f|X_f] - \hat{Y}_f \\&= \varepsilon_f + (\mathbb{E}[Y_f|X_f] - \hat{Y}_f) \\&= \varepsilon_f + (\beta_0 - b_0) + (\beta_1 - b_1)X_f.\end{aligned}$$

Prediction

From the sampling distributions derived earlier, $\text{var}(\hat{Y}_f)$ is

$$\begin{aligned}\text{var}(b_0 + b_1 X_f) &= \text{var}(b_0) + X_f^2 \text{var}(b_1) + 2X_f \text{cov}(b_0, b_1) \\ &= \sigma^2 \left[\frac{1}{n} + \frac{(X_f - \bar{X})^2}{(n-1)s_x^2} \right]\end{aligned}$$

And hence the variance of our predictive error is

$$\text{var}(e_f) = \sigma^2 \left[\textcolor{red}{1} + \frac{1}{n} + \frac{(X_f - \bar{X})^2}{(n-1)s_x^2} \right]$$

Prediction

Putting it all together, we have that

$$Y_f \sim N \left(\hat{Y}_f, \sigma^2 \left[1 + \frac{1}{n} + \frac{(X_f - \bar{X})^2}{(n-1)s_x^2} \right] \right)$$

(sums of normals are normal) or, with estimated variance,

$$Y_f \sim t_{n-p} \left(\hat{Y}_f, s^2 \left[1 + \frac{1}{n} + \frac{(X_f - \bar{X})^2}{(n-1)s_x^2} \right] \right).$$

Finally, a $(1 - \alpha)100\%$ confidence interval for Y_f is thus

$$b_0 + b_1 X_f \pm t_{n-2, \alpha/2} \left(s \sqrt{1 + \frac{1}{n} + \frac{(X_f - \bar{X})^2}{(n-1)s_x^2}} \right)$$

Prediction

Looking closer at what we'll call

$$s_{pred} = s \sqrt{1 + \frac{1}{n} + \frac{(X_f - \bar{X})^2}{(n-1)s_x^2}}.$$

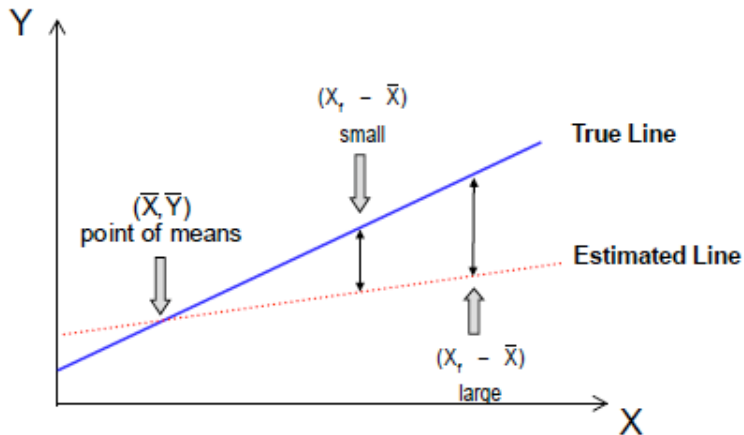
A large predictive error variance (high uncertainty) comes from

- ▶ Large s (i.e., large ε 's).
- ▶ Small n (not enough data).
- ▶ Small s_x (not enough observed spread in covariates).
- ▶ Large $(X_f - \bar{X})$.

The first three are familiar... what about the last one?

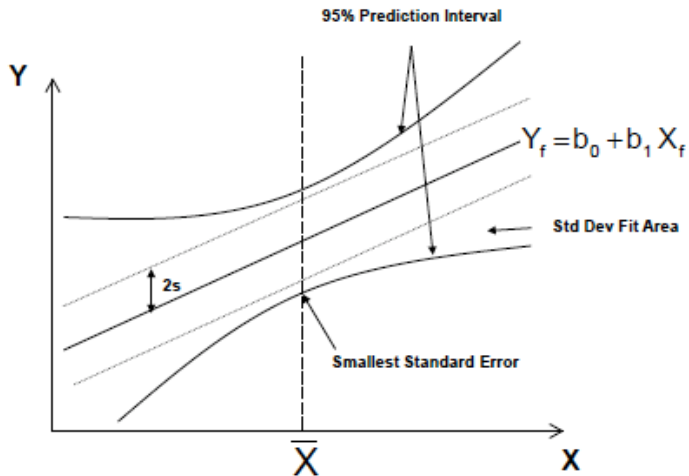
Prediction

For X_f far from our $\bar{X}$, the space between lines is magnified...



Prediction

⇒ The prediction interval needs to **widen away from $\bar{X}$**



Prediction in Python

```
> Xf = pd.DataFrame('SPX': [0.75])
> predictions = model_capm.get_prediction(Xf)
> pred_summary = predictions.summary_frame(alpha=0.05)
> pred_summary
```

	mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
	1.11126	0.177342	0.759502	1.463018	-2.071969	4.294489

```
> s = np.sqrt(model_capm.scale)
> mu = np.mean(capm['SPX'])
> s_fit = s * np.sqrt(1/n + (Xf['SPX'] - mu)**2 / ((n-1) * np.var(capm['SPX'], ddof=1)))
s_fit: 0.177
```

```
> fit = model_capm.params['Intercept'] + model_capm.params['SPX'] * Xf['SPX']
Fitted Value: 1.11126
```

```
> s_pred = np.sqrt(s**2 + s_fit**2)
> lower_bound = fit - t_critical * s_pred
> upper_bound = fit + t_critical * s_pred
95% Prediction Interval: [-2.07197, 4.29449]
```

Notice that $s_{pred} = \sqrt{s^2 + s_{fit}^2}$; you need to square before summing.